Assembly Project Report: To-Do List Tracker

Designed by: Artin Abbaspour **Implemented by:** Student

1. Command Implementations and Control Flow

- **`ADD` (`add_task.asm`)**: Parses priority markers (`!`, `*`, `.`) and the quoted description. It opens `todo.txt`, scans the last line backwards to extract the most recent ID, increments it, and formats the new task (`ID|P|PRIORITY|DESC\n`). Finally, it recalculates the string length to safely append it to the database.
- **`LIST` (`list_tasks.asm`)**: Iterates through `todo.txt` line by line. It tokenizes each line using the `|` delimiter, maps the internal `P`/`D` statuses to visual `[ ]`/`[X]` checkboxes, and prints the result to the console.
- **`DONE` & `EDIT` (`mark_done.asm`, `edit_task.asm`)**: Both routines parse the target ID and load the entire file into memory. They iterate over the lines, checking if the current line's ID matches the target. If it matches, they update the status or description in the output buffer. If it does not match, they securely copy the ID and remaining fields to preserve the existing task. The file is then overwritten with the new buffer.
- **`DELETE` (`delete_task.asm`)**: Similar to `EDIT`, it reads the file line by line. When it finds the line matching the target ID, it simply skips copying it to the output buffer, effectively removing it when the file is rewritten.
- **`CLEAR` (`clear_tasks.asm`)**: Opens `todo.txt` using the `O_TRUNC` flag. This truncates the file size to 0 bytes, instantly wiping the database without needing to process lines.
- **`ARCHIVE` (`archive_tasks.asm`)**: Scans `todo.txt` for any task with the `D` status. It copies these lines to an append-only `archive.txt` file, and rebuilds `todo.txt` using only the pending (`P`) lines.
- **`STATS` (`stats.asm`)**: Iterates over the tasks to accumulate counts of total, pending, and completed items. It then computes the integer completion percentage `(completed * 100) / total` and displays the metrics.
- **`SORT` (`sort_tasks.asm`)**: Loads tasks into memory and performs an in-place reordering based on the requested column (Status: `P` before `D`; or Priority: `!` > `*` > `.`), rewriting the sorted list back to disk.

2. Additional Helper Routines (`lib/string_utils.asm`)

To facilitate easier command parsing, several helper routines were added: * **`skip_word`**: Advances the string pointer past a contiguous block of non-space characters and skips any trailing spaces. * **`atoi`**: Parses a null-terminated string representation of a number and converts it into a 64-bit unsigned integer (`rax`). * **`itoa`**: Converts a 64-bit unsigned integer into an ASCII string representation. * **`GetStrlen`**: Scans a string for its null-terminator to dynamically return its byte length (`rdx`). * **`newLine`**: A quick utility to safely output a single `\n` character to the console.

3. Important Variables in `section .data`

- **`msg_*_success`**: Confirmation messages (e.g., "Task added", "Task deleted") displayed upon successful command execution.
- **`msg_*_not_found`**: Standardized error messages alerting the user that the provided ID does not exist in the database.
- **`msg_*_syntax_err`**: Error messages triggered when the user types a command with malformed arguments (e.g., missing quotes).

4. Important Variables in `section .bss`

- **`*_in_buf` / `*_out_buf`**: Large 8192-byte static arrays used to load the entire `todo.txt` into memory for reading, and to construct the updated file contents before performing a bulk write.
- **`*_tgt_id` / `*_cur_id`**: 32-byte buffers used to hold string-representations of the target ID requested by the user and the current ID being parsed from the file during search loops.

5. Design Choices & Edge Cases

- **File Rewriting Strategy**: Instead of attempting complex in-place binary edits (which is extremely difficult for variable-length lines in Assembly), commands like `EDIT`, `DELETE`, and `DONE` pull the entire database into `in_buf`, build the modified result sequentially in `out_buf`, and then completely overwrite the file.
- **Handling Malformed Lines**: The line parsers are built with `.malformed_copy` fallback blocks. If a line lacks proper `|` delimiters or contains unexpected formats, the program gracefully copies the line as-is rather than crashing or deleting user data.
- **Overcoming Syscall Clobbering**: The provided `appendFile` wrapper inherently modifies the `rdx` register to pass file creation permissions (0644). To prevent writing massive amounts of trailing null bytes, `add_task.asm` dynamically recalculates the exact string length using `GetStrlen` *after* calling `appendFile` and immediately before calling `writeFile`.